AI ASSISTED CODING

ASSIGNMENT-4.1

Name: M. Shashi

HtNo: 2303A51028

Q1. Zero-Shot Prompting (Basic Lab Task)

Task:

Write a Python function that classifies a given text as Spam or Not Spam using zero-shot prompting.

Steps:

1. Construct a prompt without any examples.

2. Clearly specify the output labels.

3. Display only the predicted label.

Input:

"Congratulations! You have won a free lottery ticket."

Expected Output:

Spam

```python
# generate a python function that classifies a given text as spam or not spam
def classify_spam(text):
    spam_keywords=["win","prize","free","click","buy now","subscribe","limited time offer"]
    text_lower=text.lower()
    for keyword in spam_keywords:
        if keyword in text_lower:
            return "Spam"
    return "Not Spam"
text=input("Enter the text message: ")
print(classify_spam(text))
```

```
ROBLEMS 1    OUTPUT    TERMINAL    PORTS

✓ TERMINAL
on/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-4.py
Enter the text message: Congratulations! You have won a free lottery ticket.
Spam
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/py
arsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-4.py
Enter the text message: Buy one course and get 2 courses for free
Spam
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/py
arsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-4.py
Enter the text message: Timetable for this academic year
Not Spam
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Q2. One-Shot Prompting (Emotion detection)

Task:

Write a Python program that detects the emotion of a sentence using one-shot prompting.

Emotions: [‘happy', 'sad', 'angry', 'excited', 'nervous', ’neutral’]

Steps:

1. Provide one labeled example inside the prompt.

2. Take a sentence as input.

3. Print the predicted emotion

```python
'''
I'm sad, you are not selected for the interview.
display emotion whether it is happy, sad, angry, excited , nervous, neutral
'''
def detect_emotion(text):
    text_lower=text.lower()
    if any(word in text_lower for word in ["happy","joy","excited","glad","pleased"]):
        return "Happy"
    elif any(word in text_lower for word in ["sad","unhappy","sorrow","depressed","down"]):
        return "Sad"
    elif any(word in text_lower for word in ["angry","mad","furious","irritated","annoyed"])
        return "Angry"
    elif any(word in text_lower for word in ["nervous","anxious","worried","tense","apprehen
        return "Nervous"
    else:
        return "Neutral"
text=input("Enter the text: ")
print(detect_emotion(text))
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Pytho
13/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-4.py
Enter the text: I'm sad, you are not selected to the next round
AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_a
ignment-4.py
Enter the text: I'm happy my dad gave me a chocolate
Happy
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Pytho
13/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-4.py
Enter the text: Hi
Neutral
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Q3. Few-Shot Prompting (Student Grading Based on Marks)

Task:

Write a Python program that predicts a student's grade based on marks using few-shot prompting.

Grades:

['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

• 90–100 → A

• 80–89 → B

• 70–79 → C

• 60–69 → D

• Below 60 → F

```
90-100
display A
80-89
display B
70-79
display C
60-69
display D
below 60
display F
'''
def grade_student(score):
    try:
        score=float(score)
    except ValueError:
        return "Invalid Input"
    if score<0 or score>100:
        return "Invalid Input"
    if 90<=score<=100:
        return "A"
    elif 80<=score<90:
        return "B"
    elif 70<=score<80:
        return "C"
    elif 60<=score<70:
        return "D"
    else:
        return "F"
score=input("Enter the score: ")
print(grade_student(score))
```

```
b_assignment-4.py
Enter the score: 20
F
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/User
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ド
b_assignment-4.py
Enter the score: 100
A
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/User
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ド
b_assignment-4.py
Enter the score: 60
D
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Q4. Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task:

Write a Python program that predicts a person's Indian Zodiac sign (Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model): The Indian Zodiac cycle starts in March with Mesha and follows this order:

March → Mesha

April → Vrishabha

May → Mithuna

June → Karka

July → Simha

August → Kanya

September → Tula

October → Vrischika

November → Dhanu

December → Makara

January → Kumbha

February → Meena

```
March
display Mesha
April
display Vrishabha
May
display Mithuna
June
display Karka
July → Simha
August → Kanya
September → Tula
October → Vrischika
November → Dhanu
December → Makara
January → Kumbha
February → Meena
'''
def zodiac_sign(month):
    month=month.lower()
    zodiac_dict={
        "march":"Mesha",
        "april":"Vrishabha",
        "may":"Mithuna",
        "june":"Karka",
        "july":"Simha",
        "august":"Kanya",
        "september":"Tula",
        "october":"Vrischika",
        "november":"Dhanu",
        "december":"Makara",
        "january":"Kumbha",
        "february":"Meena"
    }
    return zodiac_dict.get(month,"Invalid Input")
month=input("Enter the month: ")
print(zodiac_sign(month))
```

```
b_assignment-4.py
Enter the month: Marsh
Invalid Input
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA
ograms/Python/Python313/python.exe c:/Users/varsh
b_assignment-4.py
Enter the month: March
Mesha
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA
ograms/Python/Python313/python.exe c:/Users/varsh
b_assignment-4.py
Enter the month: February
Meena
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA
```

Q5. Result Analysis Based on Marks

Task: Write a Python program that determines whether a student Passes or Fails based on marks using Chain-of-Thought (CoT) prompting.

Result Categories: ['Pass', 'Fail']

```python
# Take list of input marks from the user
# Verify whether the marks are within a valid range
# Compare marks if it is greater than or equal to 40
#Output list of pass or fail for each mark
marks_input=input("Enter marks separated by commas: ").split(',')
results=[]
for mark in marks_input:
    mark=mark.strip()
    try:
        mark=float(mark)
    except ValueError:
        results.append("Invalid Input")
        continue
    if 0<=mark<=100:
        if mark>=40:
            results.append("Pass")
        else:
            results.append("Fail")
    else:
        results.append("Invalid Input")
print("Results:",results)
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL Python: lab

```
b4.py
Enter marks separated by commas: 10,50,30,49,100
Results: ['Fail', 'Pass', 'Fail', 'Pass', 'Pass']
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Q6 Voting Eligibility Check (Chain-of-Thought Prompting)

Task: Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting.

```python
# Take the input age from the user
# check whether the person is eligible to vote or not
# output eligible or not eligible
age_input=input("Enter your age: ")
try:
    age=int(age_input)
except ValueError:
    print("Invalid Input")
    exit()
if age>=18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

PROBLEMS 1 OUTPUT TERMINAL PORTS

TERMINAL Python: lab

```
Enter your age: 18
Eligible to vote
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/thir
b4.py
Enter your age: 1
Not eligible to vote
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/thir
b4.py
Enter your age: 0
Not eligible to vote
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData
ograms/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/thir
b4.py
Enter your age: hi
Invalid Input
```

Q7 Prompt Chaining (String Processing – Palindrome Names) Task: Write a Python program that uses the prompt chaining technique to identify palindrome names from a list of student names.

```python
# Generate a list of student names and store it in a variable
# From the generated list generate a list which are palindrome names only'
# Output the palindrome names list
student_names=["Anna","Bob","Cathy","David","Eve","Hannah","Ian","Otto"]
palindrome_names=[]
for name in student_names:
    if name.lower()==name.lower()[::-1]:
        palindrome_names.append(name)
print("Palindrome names:",palindrome_names)
```

PROBLEMS 1 OUTPUT TERMINAL PORTS ...

TERMINAL

```
Palindrome names: ['Anna', 'Bob', 'Eve', 'Hannah', 'Otto']
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Q8 Prompt Chaining (String Processing – Word Length Analysis)

Task: Write a Python program that uses prompt chaining to analyze a list of words. In the first prompt, generate a list of words. In the second prompt, traverse the list and calculate the length of each word. In the third prompt, use the output of the previous step to determine whether each word is Short (length less than 5) or Long (length greater than or equal to 5), and display the result foreach word

```python
#Prompt chaining
# Generate a list of words
# Traverse the list and calculate the length of each word
# use the lengths to determine whether each word is short (length<5), long
# display result of each word as short or long
words_list=["apple","bat","cat","dolphin","elephant","fig","goat"]
word_lengths=[]
for word in words_list:
    length=len(word)
    word_lengths.append(length)
word_types=[]
for length in word_lengths:
    if length<5:
        word_types.append("Short")
    else:
        word_types.append("Long")
for i in range(len(words_list)):
    print(f"{words_list[i]}: {word_types[i]}")
```

ROBLEMS 1 OUTPUT TERMINAL PORTS ...

TERMINAL

```
apple: Long
bat: Short
cat: Short
dolphin: Long
elephant: Long
fig: Short
goat: Short
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```